

SafeSignal

A verified voice that travels faster than the rumour

Document	Product Requirements Document
Product	SafeSignal
Version	2.0
Date	22 September 2026
Status	Final for hackathon submission
Classification	Public

CONTENTS

- 1. The problem
- 2. Stated assumptions
- 3. The principle
- 4. Users and roles
- 5. How it works
- 6. Where the AI sits
- 7. System architecture
- 8. Distribution
- 9. Scope for the hackathon
- 10. Risks and open questions
- 11. What success looks like

1. The problem

It is 6:40 in the evening. Amara is closing her shop and must decide whether the road home is safe. A cousin has forwarded a WhatsApp message. A neighbour saw movement on the road. The local vigilante group has a radio but no way to reach everyone quickly.

By the time real information reaches her, it is either too late or it is a rumour that has already sent the market into panic for nothing.

The brief names three failures, and they pull against each other:

Too much noise. Rumours, forwards and second-hand accounts arrive constantly, and nothing separates them.

Too little verified signal. Information that is checked arrives slowly, because checking takes time.

No time. A report next week is worthless. Amara needs an answer in minutes.

There is a fourth problem underneath these, and it is not the same as accuracy: **trust**. A false alarm that empties the market teaches people to ignore the next real warning.

The insight this product is built on

The community is not short of knowledge. Somebody in that town already knows whether the road is safe. The vigilante group has a radio but no way to reach everyone quickly — that is a **distribution problem, not a knowledge problem**.

So the product's job is not to investigate. It is to **route and confirm what is already known**.

2. Stated assumptions

The brief is short, and it would be easy to smuggle in assumptions and present them as findings. These are separated deliberately.

What the brief tells us: there is a shop owner named Amara; there is a WhatsApp rumour, a neighbour's sighting and a vigilante radio; information is either late or unverified; she needs to know now and she needs to trust it.

What we are assuming, and why:

First, that verifiable information already exists inside the community, because the brief describes people who saw things and a vigilante group with a radio. If this is wrong, the product needs an investigation capability it does not currently have.

Second, that the community can designate people it already trusts. The brief does **not** say the vigilante group is official, accountable or trusted, so the product never assumes that. It designs for a **role** — a trusted local verifier — and lets each community decide who fills it.

Third, that most people have a phone that receives SMS. If this is wrong, the distribution layer changes but the architecture does not.

The brief mentions no language barrier, so translation is deliberately **not** treated as a core feature.

3. The principle

The AI is the amplifier, not the authority.

Truth originates from a human the community already trusts. The AI does the work humans are slow at: turning a rushed report into a clear message, deciding who needs it, and delivering it everywhere at once.

This is a deliberate rejection of the obvious approach. An AI trained on the community's own rumour flow would be learning from unverified material and would degrade toward confident nonsense. A trusted human voice beats manufactured authority.

The line that captures it: *we did not use AI to decide what is true; we used it to make the truth travel faster than the rumour.*

4. Users and roles

The trusted voice. A person or small group the community already relies on — a vigilante liaison, ward head, transport union representative or market leader. They speak or type one rushed report. They are the only source of verified truth in the system.

The resident. Amara, and everyone like her. Receives alerts without signing up for anything, and can check a rumour when one reaches her.

The coordinator. Optional, for larger deployments. Covers gaps when the primary trusted voice is unavailable, particularly at night.

5. How it works

Flow one: the trusted voice broadcasts

Something happens. The trusted voice speaks into the app, because speaking is fastest under pressure — no typing, no forms, no menus. They say what they know in whatever order it comes out.

The AI turns that rushed, messy report into a clear, structured, actionable message: what happened, where, what to do. It then decides distribution. Within seconds the message reaches everyone in the affected area.

Nobody has to ask first. Nobody has to have joined anything.

☐ Person ☒ AI component ☐ Data / infrastructure

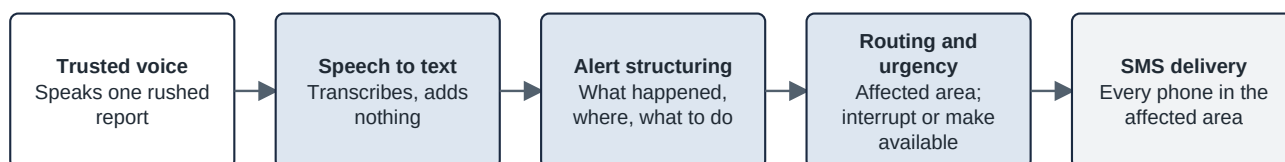


Figure 1. Flow one: from a spoken report to an alert in the right hands.

Flow two: Amara checks a rumour

A message reaches Amara and she does not know whether to believe it. She sends it into the app and gets one of three honest answers.

It matches a verified alert. The rumour is real, and here is the accurate version.

It contradicts a verified alert. It is false, or it is describing something that has since been resolved.

Nothing verified exists about it yet. This is the delicate one. The system does **not** say the road is fine. It states plainly what is known and what is not, gives safe general guidance — wait, travel with others, ask before setting out — and provides the contact details of the trusted person responsible for that specific road, chosen by location rather than offered as a generic list. It then promises to message her the moment a verified answer exists, and keeps that promise.

There is a second benefit here. A flood of people checking the same rumour tells the trusted voice exactly what the town is panicking about right now, so they know what to address next. The same is true of repeated calls to the contact number. The checking behaviour itself becomes an early warning signal.

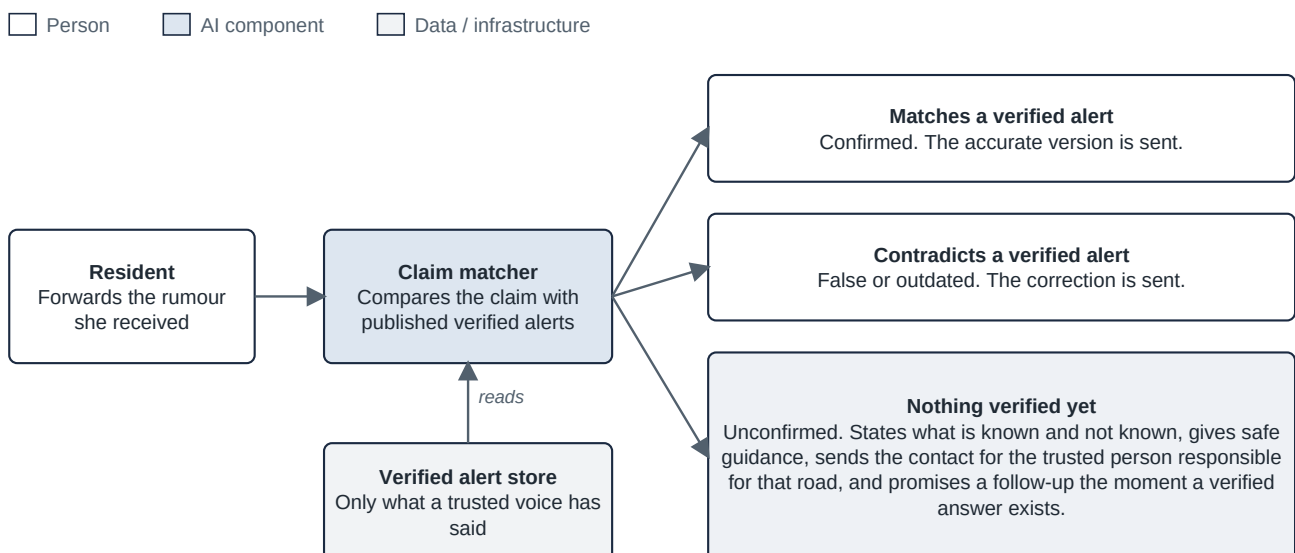


Figure 2. Flow two: the rumour check and its three honest outcomes.

6. Where the AI sits

The AI does four jobs.

It listens. It converts a spoken, rushed, unstructured report into a clear written alert — what, where, what to do — without adding anything the speaker did not say.

It routes by geography. It reads the location out of the message and works out who is genuinely affected, so a warning about one road does not alarm an entire region.

It judges urgency. It decides whether to interrupt people or simply make the information available. A town should not be woken at two in the morning for something that can wait until sunrise.

It matches claims. On the checking side, it compares an incoming rumour against what the trusted voice has already published, and reports agreement, contradiction or silence.

The job it never does is decide what is true.

This split is what makes the AI defensible. A wrong routing or urgency decision costs attention. A wrong truth judgement costs lives. Only the first kind of mistake is possible here, and the truth judgement stays with the human.

7. System architecture

Figure 3 shows the components and how information moves between them. There are two paths through the system, and they share a single source of truth: the verified alert store. Nothing enters that store except what a trusted voice has said.

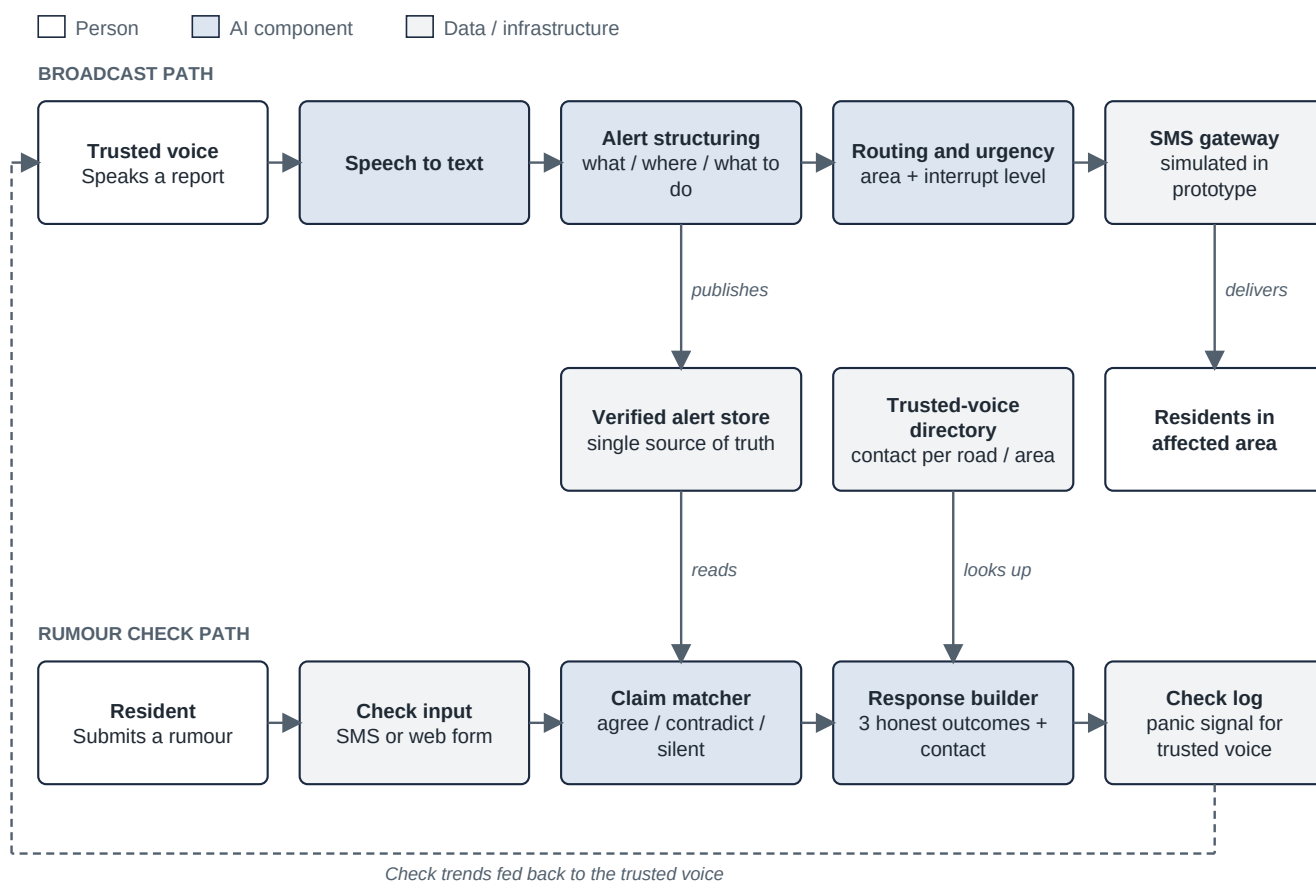


Figure 3. System architecture. Both paths share one source of truth: the verified alert store.

Speech to text. Converts the trusted voice's spoken report into text. It transcribes only; it does not interpret.

Alert structuring. Turns the transcript into a fixed format of what happened, where, and what to do. It may reorganise and clarify, but it may not add facts the speaker did not state.

Routing and urgency. Resolves the location in the alert to an affected area, selects recipients, and decides whether the alert interrupts people immediately or is made available quietly.

Verified alert store. Holds every published alert with its time, area and source. It is the only thing the claim matcher is allowed to check against.

Trusted-voice directory. Maps each road or area to the trusted person responsible for it, so the unconfirmed outcome can give a specific contact rather than a generic list.

Claim matcher and response builder. Compare an incoming rumour with the store and return one of three outcomes: matches, contradicts, or nothing verified yet. The response never states that a road is safe unless a verified alert says so.

Check log. Records every rumour check by topic and area. Clusters of checks show the trusted voice what the town is worried about right now.

SMS gateway. Delivers alerts and replies. In the hackathon prototype this is a simulated delivery view, which demonstrates the same logic without a telecom integration.

8. Distribution

Output is text, not audio. Audio is slow to consume, cannot be scanned, cannot be re-read, and cannot be checked silently in a noisy market.

The primary channel is SMS. It reaches every phone, including basic ones. No app, no data connection, no prior signup, no account. A message waits in the pocket until it is read.

Radio is not bad — it is one-to-few. You must be near a set and listening at that moment. SMS is one-to-all.

The AI selects channels per recipient where more than one exists: a trader with a basic phone, a driver already on the road and someone at home are not best reached the same way.

9. Scope for the hackathon

The brief asks for one sharp thing, not the whole platform. **The sharp thing is the path from a spoken report to a clear alert in the right hands, plus the rumour check.** That is the part nothing else does, and it is the part that makes the argument.

Built: the trusted voice speaks; the AI structures the alert; the AI decides who is affected and how urgent it is; the alert is delivered; a resident submits a rumour and receives one of the three honest answers, including the contact fallback.

Deliberately left out, with reasons: real telecom SMS integration, because a simulated delivery view demonstrates the identical logic; WhatsApp group ingestion, because those groups are end-to-end encrypted and it is not feasible; a separate production verifier application, because the trusted voice step can be simulated; multi-language support, because the brief does not describe a language barrier; and accounts, onboarding and authentication, because they demonstrate nothing about the core idea.

The demonstration runs on seeded data so it works reliably every time.

10. Risks and open questions

Single point of failure. If the trusted voice is slow, silent or wrong, the system fails — and worse, the community has been taught to wait for them. Night coverage and a deputy arrangement are the obvious mitigations, and both are unresolved.

Silence versus explicit uncertainty. When the trusted source has not spoken, the system must say "nothing confirmed yet" rather than nothing at all. Saying nothing gets read as safety.

Alert fatigue. Every unnecessary interruption reduces the weight of the next real one. This is why urgency judgement exists, and it needs measuring in the field.

Protecting the trusted voice. A person publicly identified as the community's verifier may become a target. This needs answering before any real deployment.

Genuine last-mile reach. Whether SMS truly reaches everyone who needs it is an assumption that requires field validation.

11. What success looks like

Amara gets a trustworthy answer in under a minute. The gap between something happening and the affected area knowing about it closes to seconds. Rumours are identified as rumours before the market empties. And over time, people stop acting on forwards and start waiting for the verified message — which is the real measure, because it means trust moved.